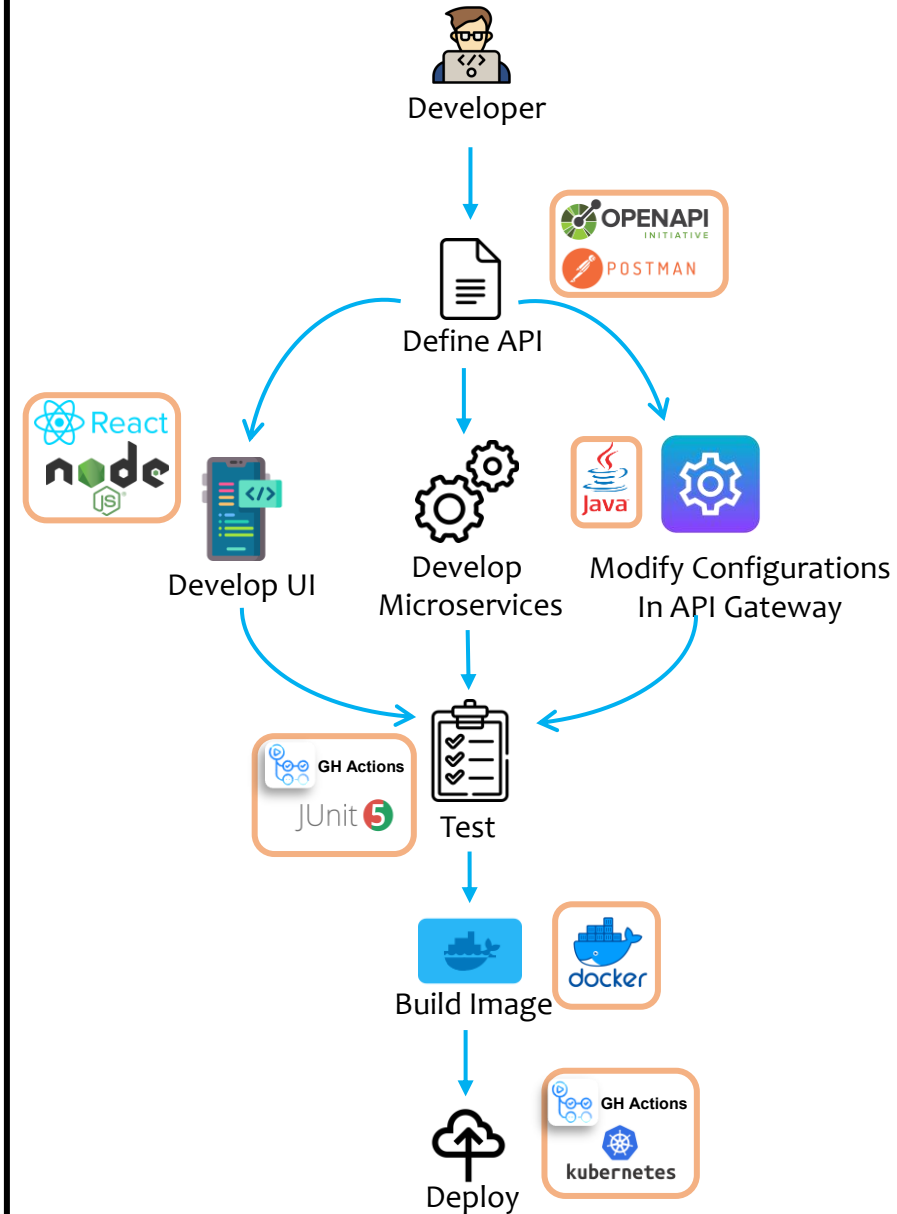


GitHub Actions

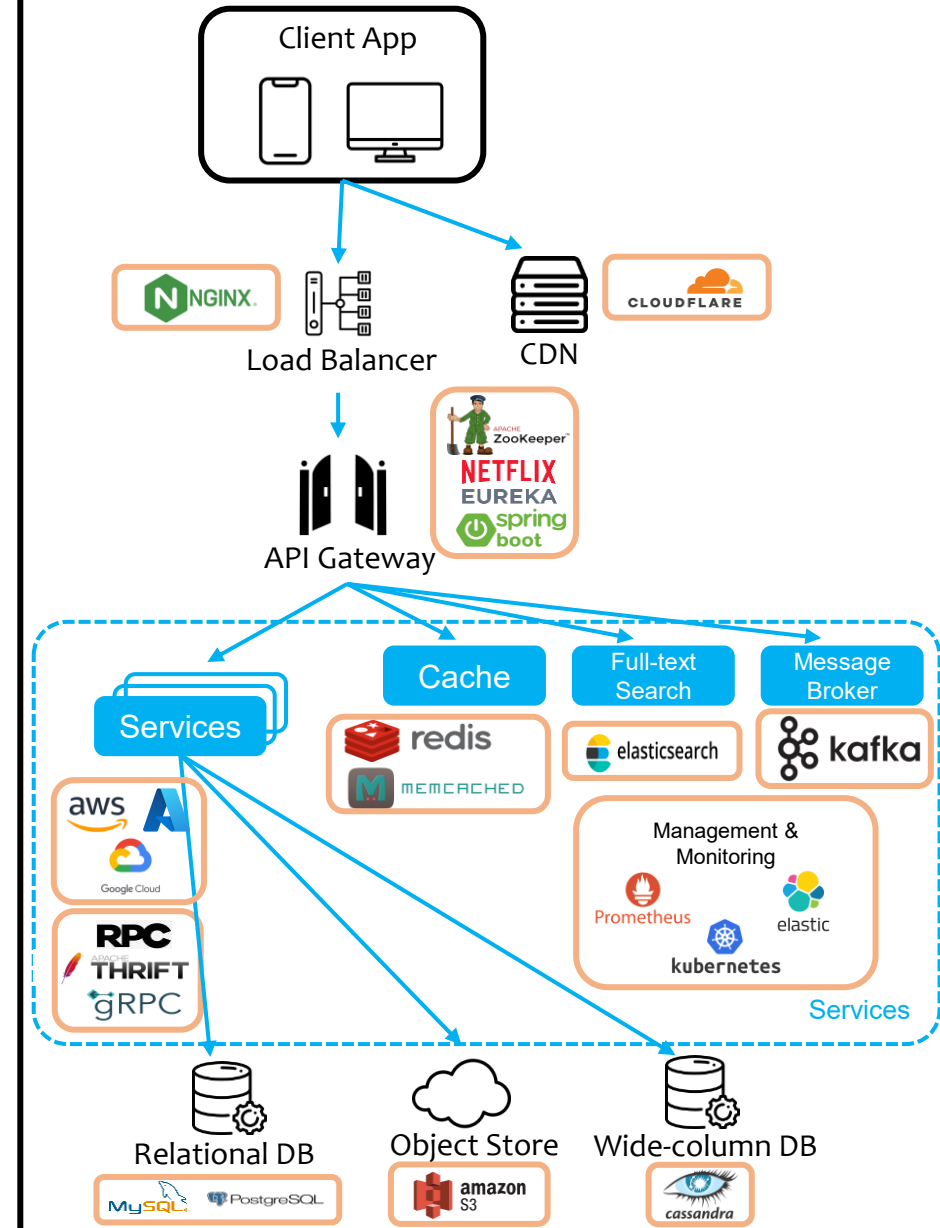
GitHub Introduction

DevOps CI/CD Implementation

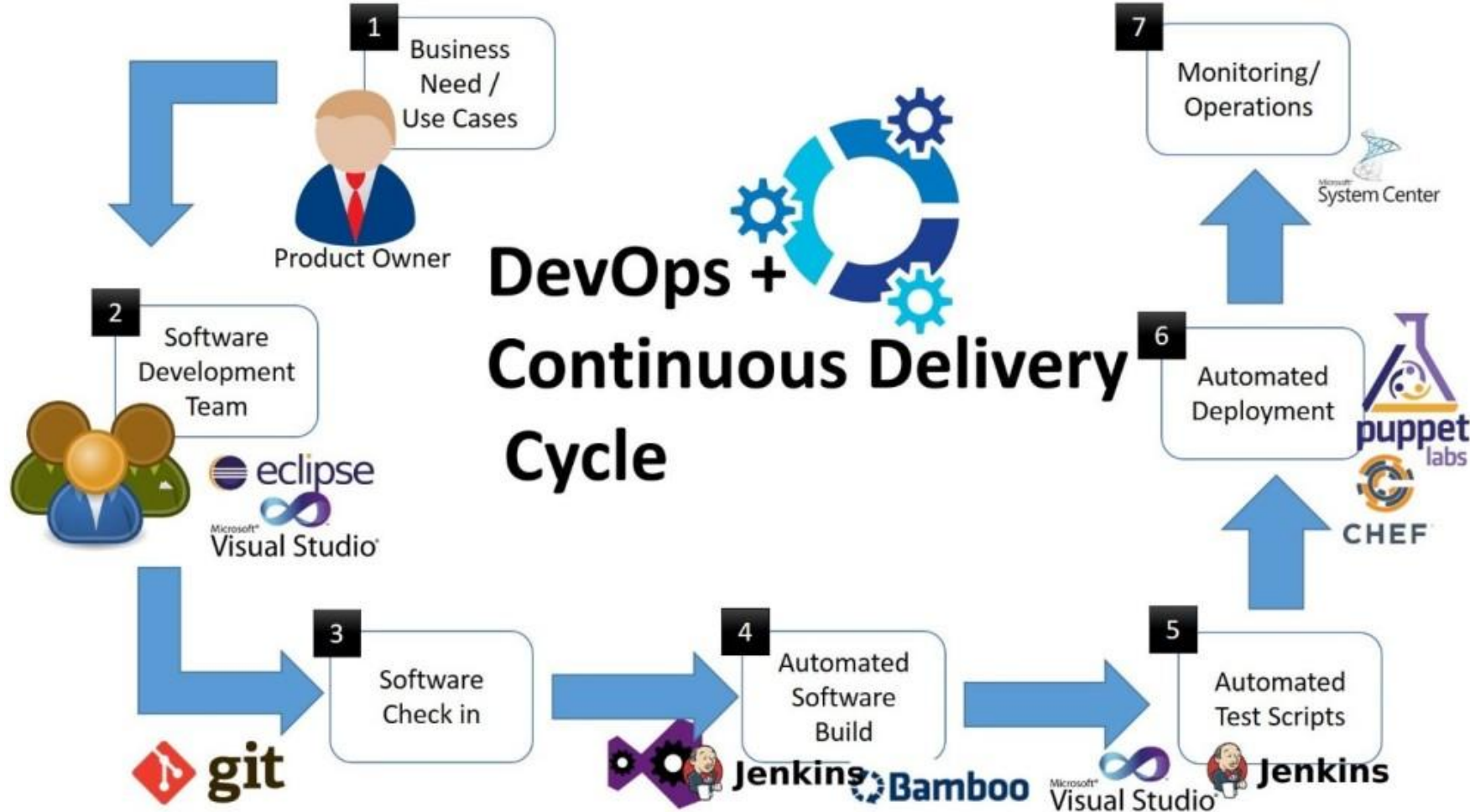
Pre - Production



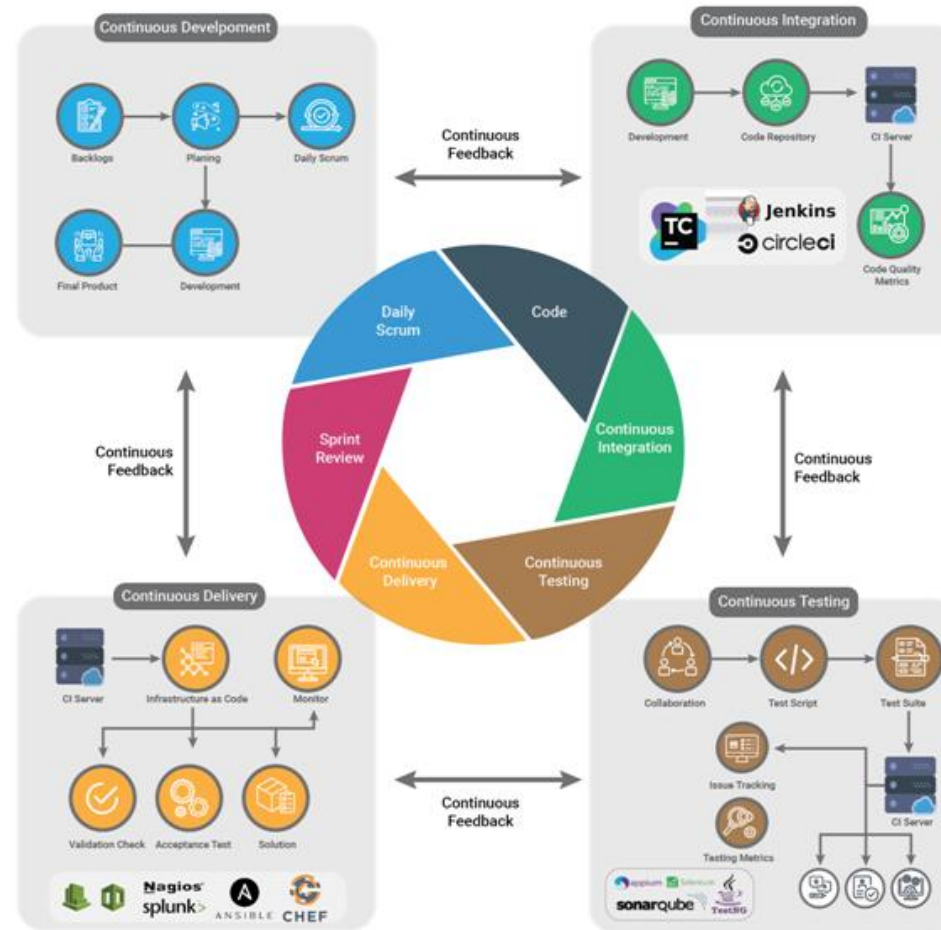
Production



DevOps Workflow



DevOps Workflow



What is GitHub Actions



- GitHub Actions is a continuous integration and continuous delivery (CI/CD) platform that allows you to automate your build, test, and deployment pipeline.
- You can create workflows that build and test every pull request to your repository, or deploy merged pull requests to production.
- GitHub Actions goes beyond just DevOps and lets you run workflows when other events happen in your repository.

What are GitHub Actions?



- Packaged workflows
- Automates common build, test, and deployment actions
- Highly configurable and extendable
- Supports an open marketplace of free and paid tools
- Includes a rich user interface system native in GitHub

Common Devops Tasks



- Add new contributors
- Close stale issues & PRs
- Pull request
- Organizational tasks: Issue created by users
- Label the issue { minor, major, reproducible, priority etc. }
- Fix the issue and raise a PR
- Verify PR and merge the code to master
- New release with a version number
- Build pipeline to test, and build code

Basic Building Blocks GH



- Workflow
- Event Triggers
- Workflow Runners
- Using Actions
- Filter Activity Types
- Contexts
- Expressions
- Variables
- Function
- Execution Flow
- Inputs
- Outputs
- Caching
- Artifacts
- Releases
- Matrices

Basic Building Blocks GH



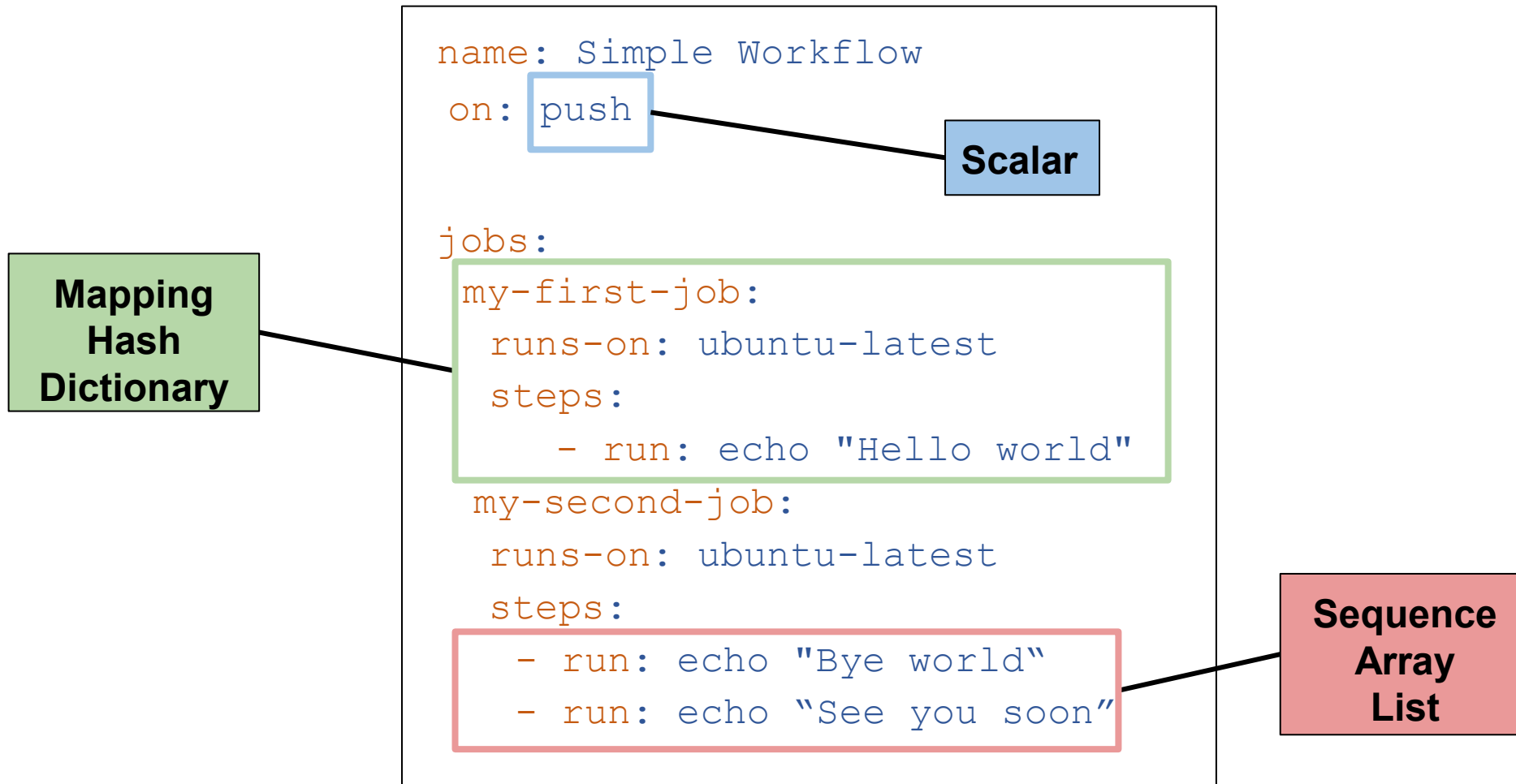
- Environments
- Custom Actions
- Reusable Workflows
- Concurrency
- Workflow Security
- Docker
- CI/CD Pipeline
- Jenkins Migration
- Bamboo Migration

Object Expression - YAML



- Files or other representations of Kubernetes Objects are generally represented in YAML.
- A “Human Friendly” data serialization standard.
- Uses white space (specifically spaces) alignment to denote ownership.
- Three basic data types:
 - mappings - hash or dictionary,
 - sequences - array or list
 - scalars - string, number, boolean etc

Object Expression - YAML



YAML vs JSON



```
name: Simple Workflow
on: push

jobs:
  my-first-job:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Hello world"
  my-second-job:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Bye world"
      - run: echo "See you soon"
```

```
{
  "name": "Simple Workflow",
  "on": "push",
  "jobs": {
    "my-first-job": {
      "runs-on": "ubuntu-latest",
      "steps": [
        {
          "run": "echo \"Hello
World\""
        }
      ]
    },
    "my-second-job": {
      "runs-on": "ubuntu-latest",
      "steps": [
        {
          "run": "echo \"Bye World\""
        }
      ]
    }
  }
}
```

Workflows



- A workflow is a configurable automated process that will run one or more jobs. Workflows are defined by a YAML file checked in to your repository and will run when triggered by an event in your repository, or they can be triggered manually, or at a defined schedule.

What are those workflows?



Platform for open-source projects

- Publically available to use and contribute to the project

Used by 420k

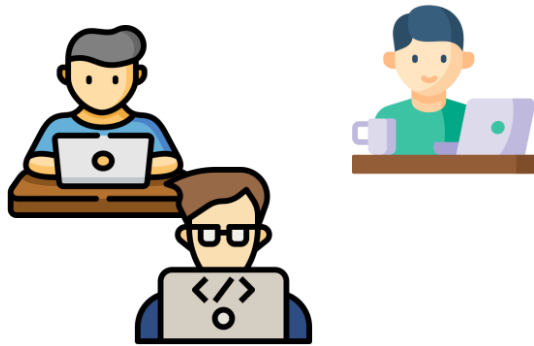


Contributors 435

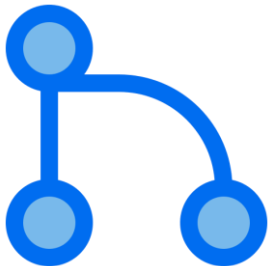
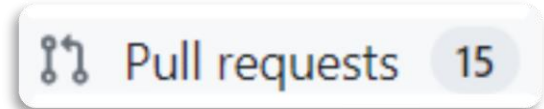


+ 421 contributors

What are those workflows?



- add new contributors
- pull requests are created
-

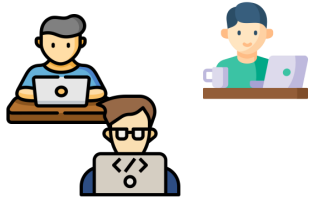


Organizational Task

What are those workflows?



Organizational Task



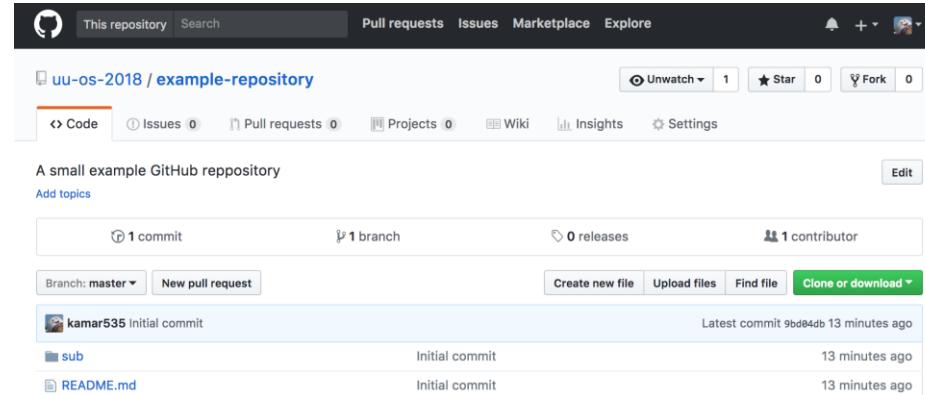
Contributors



Users



New Issue

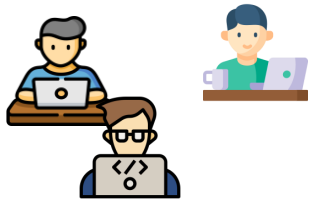


18
New issues

What are those workflows?



Organizational Task

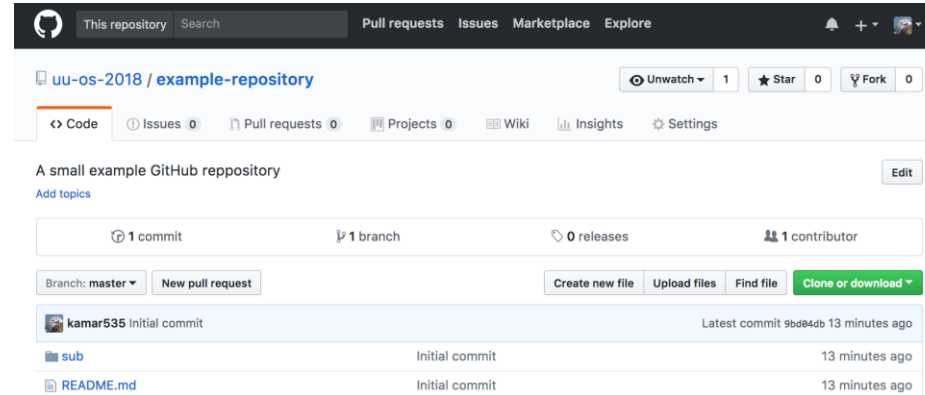


Contributors

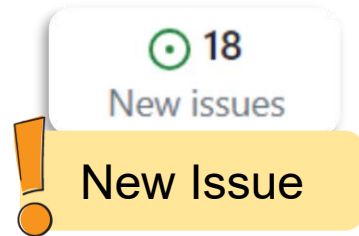


Users

GitHub Actions



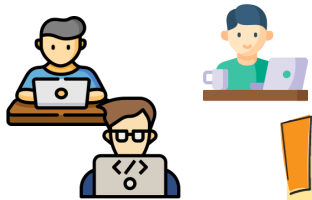
- is it major / minor?
- is it reproducible?
- assign to a contributor



What are those workflows?



Organizational Task



Contributors

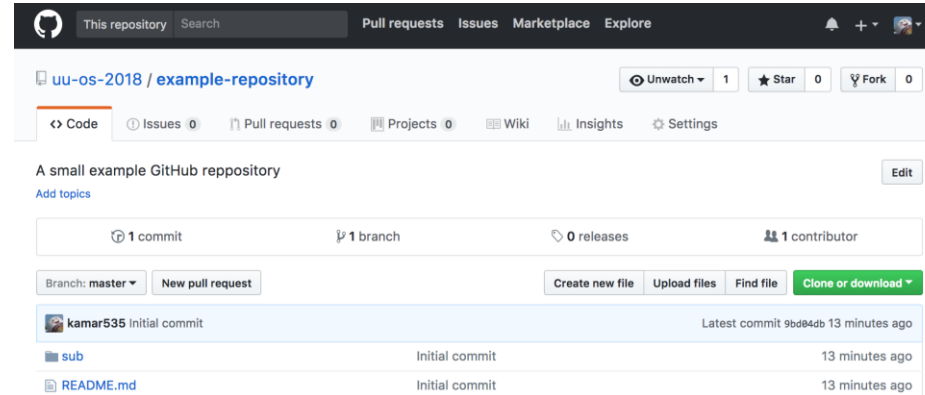


New Issue



Users

GitHub Actions



18
New issues

- is it major / minor?
- is it reproducible?
- assign to a contributor

What are those workflows?



Organizational Task



Contributors

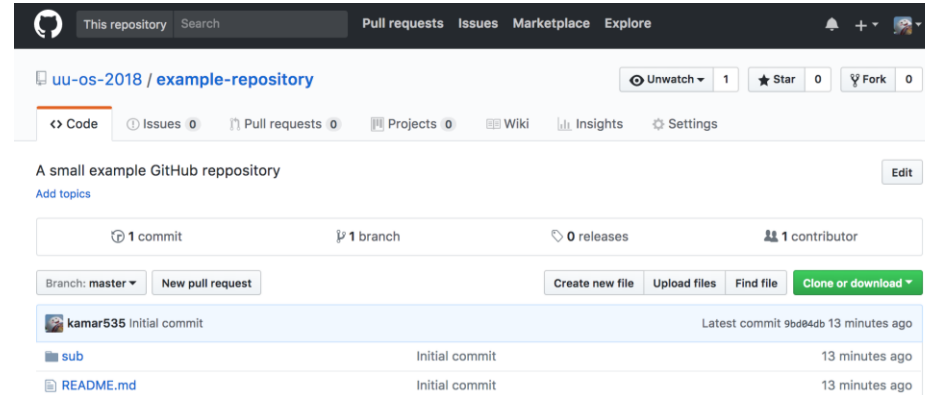


New Issue



Users

GitHub Actions



18
New issues

What are those workflows?



Organizational Task



Contributors

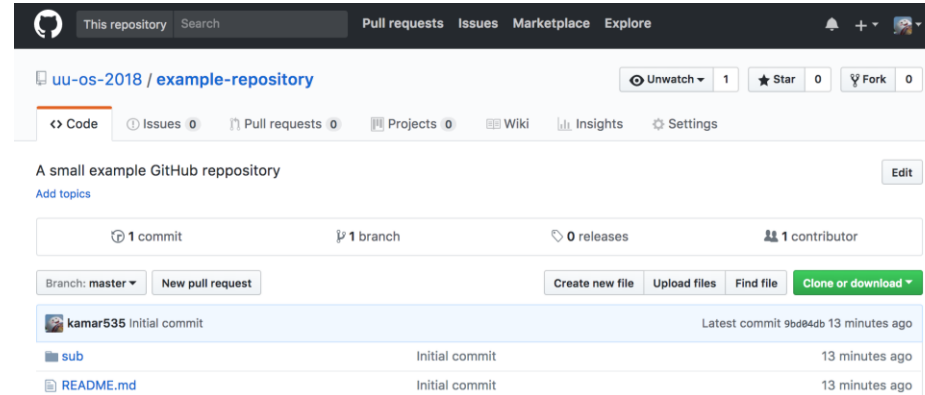


New Issue



Users

GitHub Actions



18

New issues



Pull requests 15

What are those workflows?



Organizational Task



Contributors

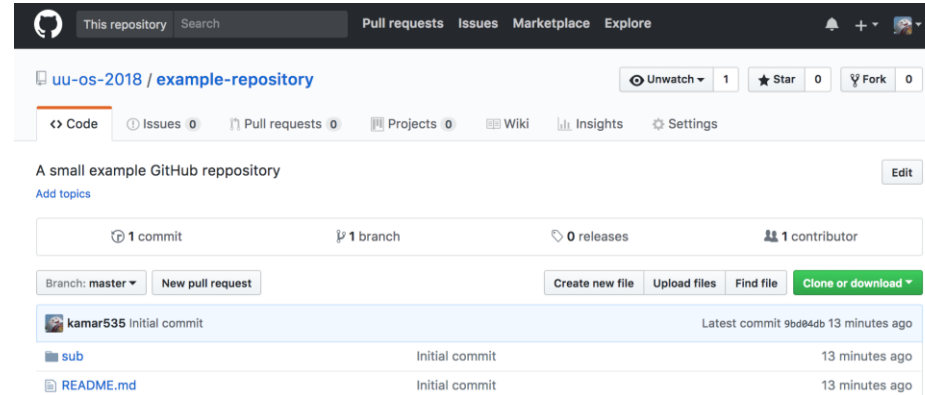


New Issue

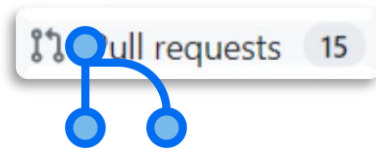
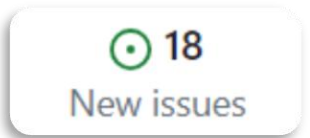


Users

GitHub Actions



- review pull request
- is the bug fixed?
- merge to master branch



What are those workflows?

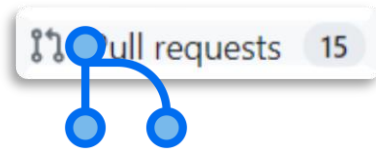
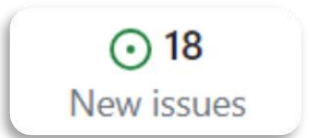
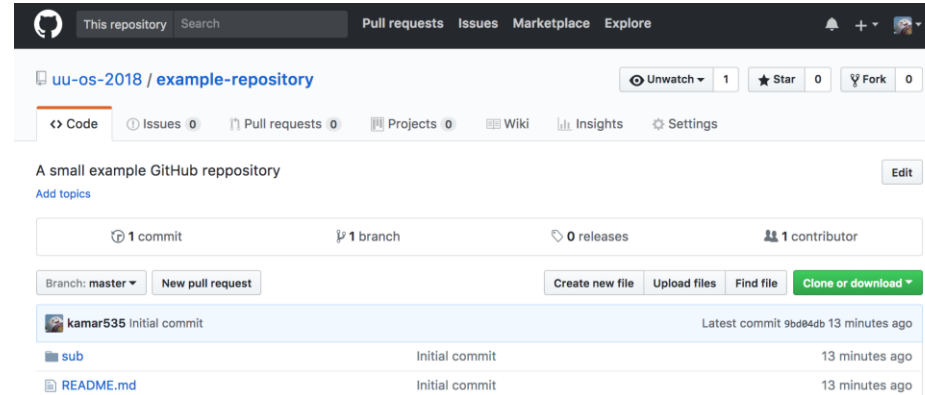


Organizational Task

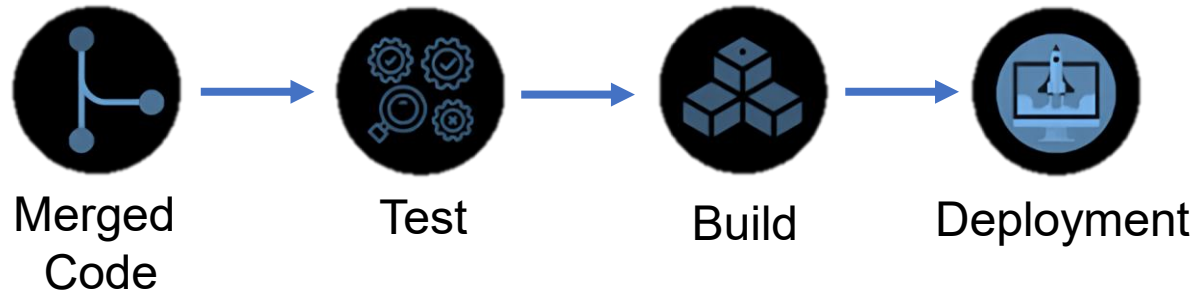


Contributors

- Prepare release notes
- Update version number



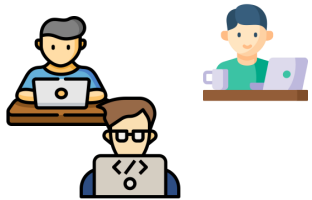
CI/CD Pipeline



What are those workflows?



Organizational Task

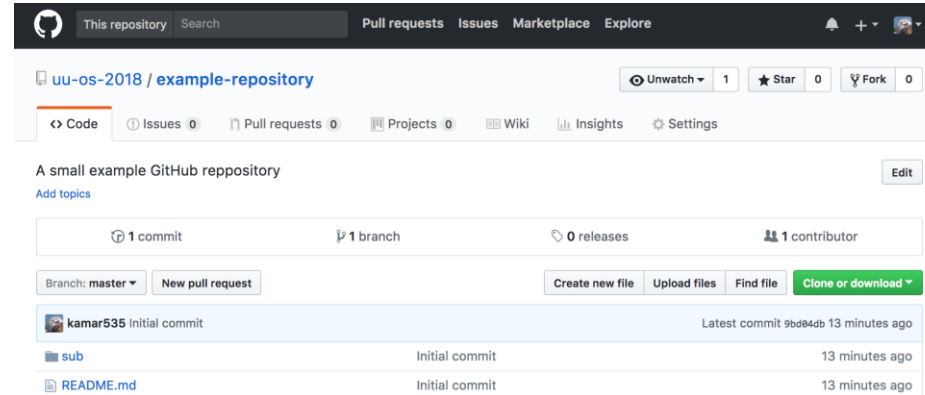


Contributors

- is it major / minor?
- is it reproducible?
- assign to a contributor

- review pull request
- is the bug fixed?
- merge to master branch

- Prepare release notes
- Update version number



18

New issues



Pull requests 15

WORKFLOWS



Merged
Code



Test



Build



Deployment

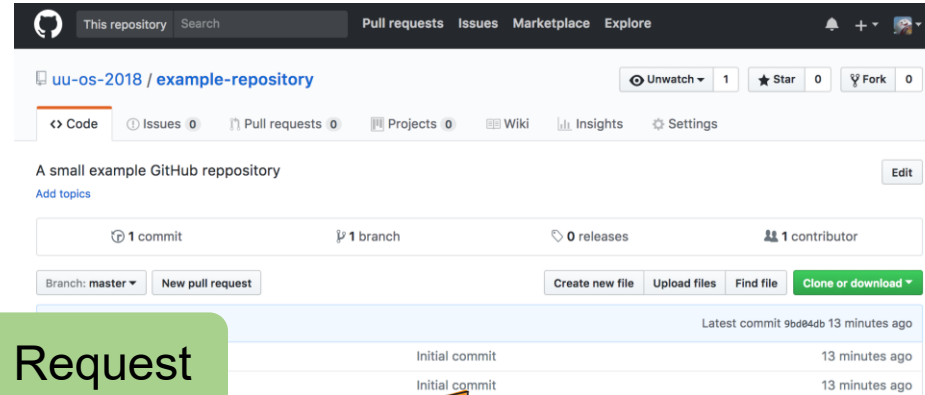
What are those workflows?



Organizational Task



Contributors



18
New issues

Pull requests 15

✓ Pull Request

- is it major / minor?
- New Issue
- assign to a contributor

! New Issue

- review pull request
- is the bug fixed?
- master branch

! New Issue

✓ Pull Request

✓ Pull Request

release notes
update version number

WORKFLOWS



Merged
Code



Test



Build



Deployment

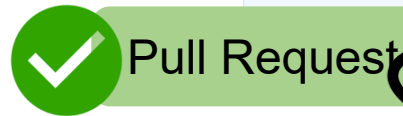
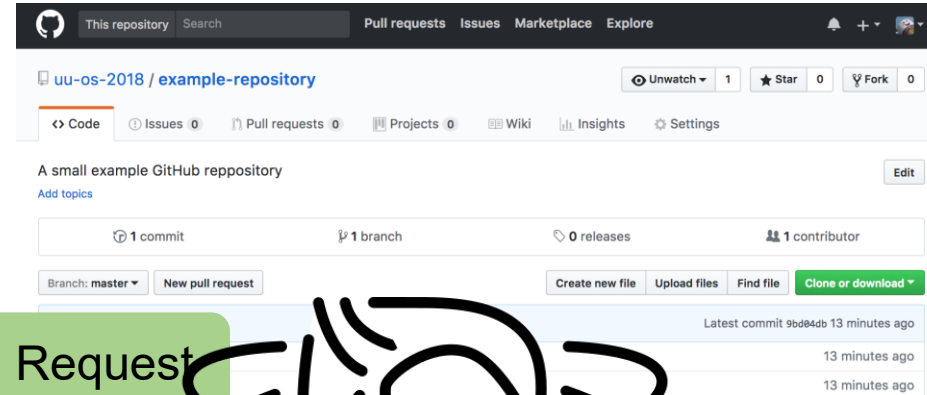
What are those workflows?



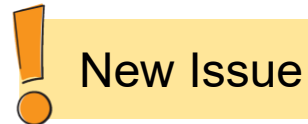
Organizational Task



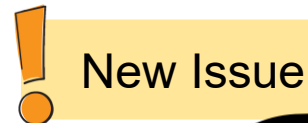
Contributors



Pull Request



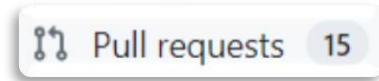
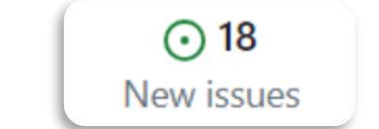
New Issue



New Issue

New Issue

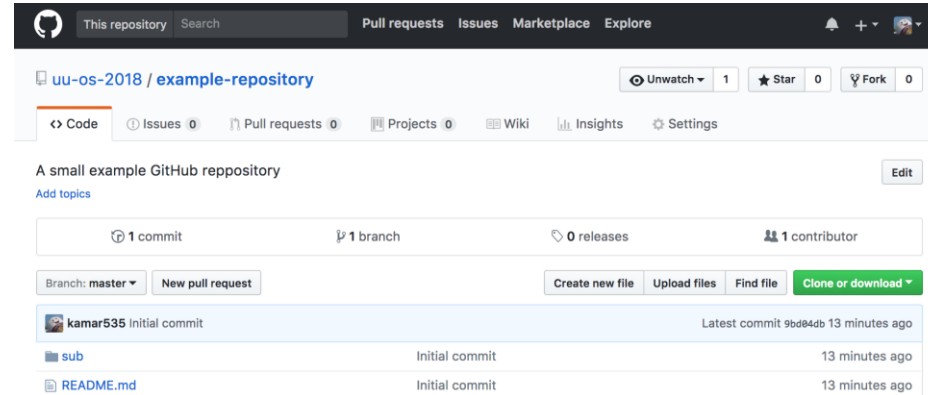
Pull Request



What are those workflows?



Organizational Task

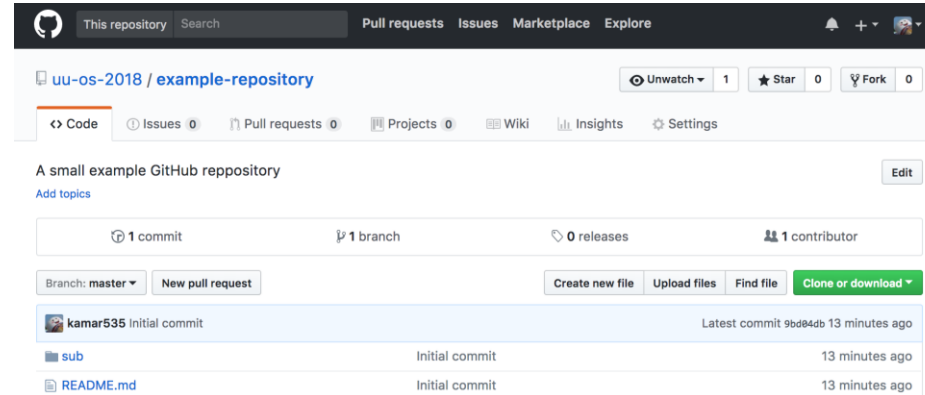


Automate as much as possible!

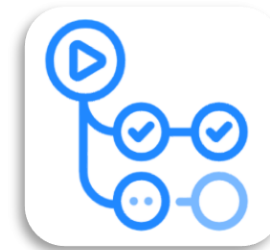
What are those workflows?



Organizational Task



Automate as much as possible!



Creating a Workflow File



- Create a **.github/workflows** directory in your repository on GitHub if this directory does not already exist.
- In the **.github/workflows** directory, create a file with the **.yml** or **.yaml** extension.

```
✓ GITHUB-DEMO
  ✓ .github \ workflows
    ! SimpleWorkflow.yaml
```

Workflows, Jobs & Steps



- **Workflows:**

- Are defined at the repository level
- Define which triggers actually start the workflow
- Are composed of one or more jobs

- **Jobs:**

- Are defined at the workflow level
- Define in which execution environment they are run
- Are composed of one or more steps
- Run in parallel by default

- **Steps:**

- Are defined at the job level
- Define the actual script or GitHub Action that will be executed
- Run sequentially by default



```
name: Simple Workflow
on: push

jobs:
  my-first-job:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Hello world"
  my-second-job:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Bye world"
```

Triggers:

You can configure your workflows to run when specific activity on GitHub happens, at a scheduled time, or when an event outside of GitHub occurs.

Simple Workflow



```
EXPLORER  ...  ! SimpleWorkflow.yaml X

✓ GITHUB-DEMO
  ✓ .github\workflows
    ! SimpleWorkflow.yaml

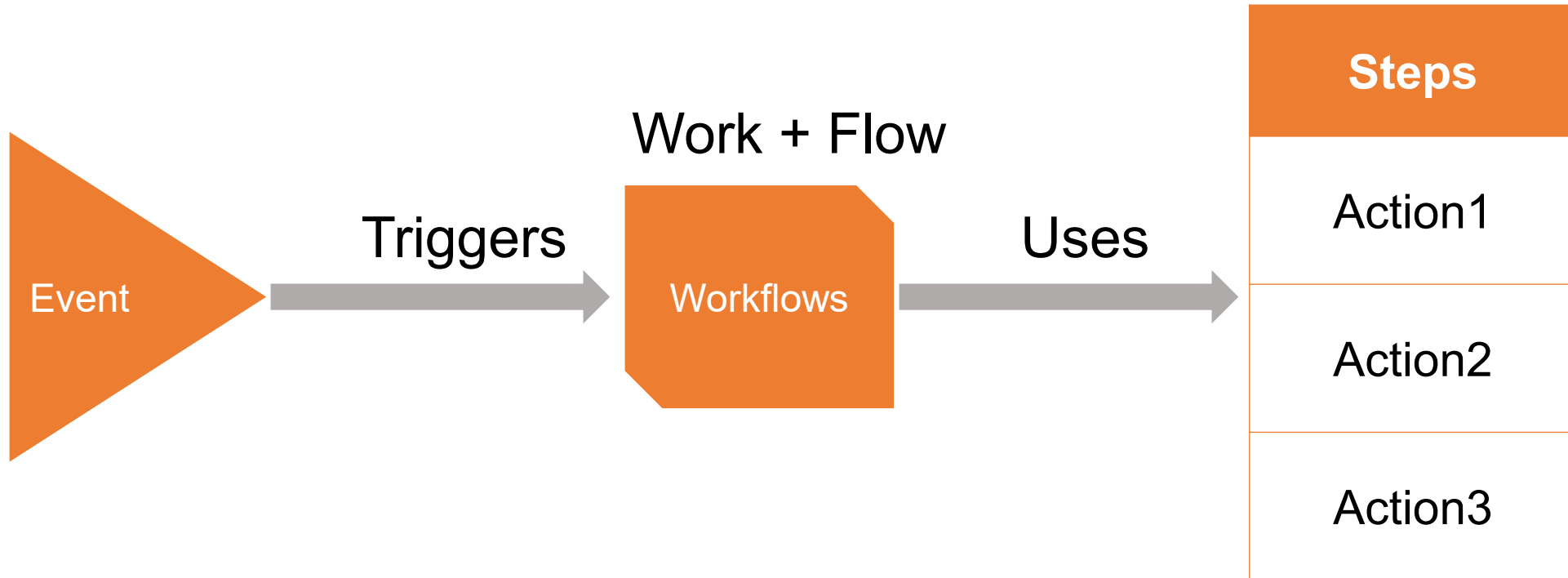
.github > workflows > ! SimpleWorkflow.yaml
1  name: Simple Workflow
2  on: push
3
4  jobs:
5    my-first-job:
6      runs-on: ubuntu-latest
7      steps:
8        - run: echo "Hello world"
9    my-second-job:
10     runs-on: ubuntu-latest
11     steps:
12       - run: echo "Bye world"
```

Test Workflow



- Since our workflow's trigger is a push event, push a commit on any repository branch and attach Triggering Push as the commit message.
- Go to the repository's page on GitHub and click the **Actions tab** under the repository's name.
- At the top, the commit message for the push that triggered the workflow will be visible.
- Clicking on the job will show the output from each step in the workflow.

GitHub Action Workflow



- GitHub Action Workflow Directory: `github/workflows/`
- File Extension: `.yml`



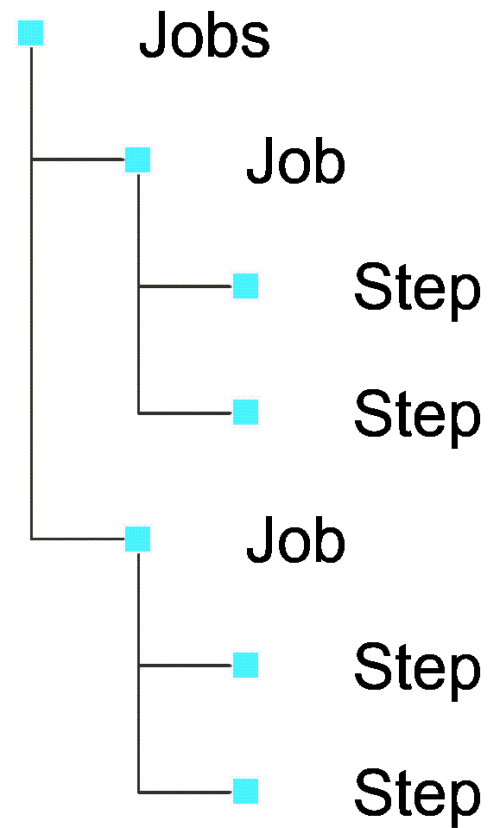
Build

Test

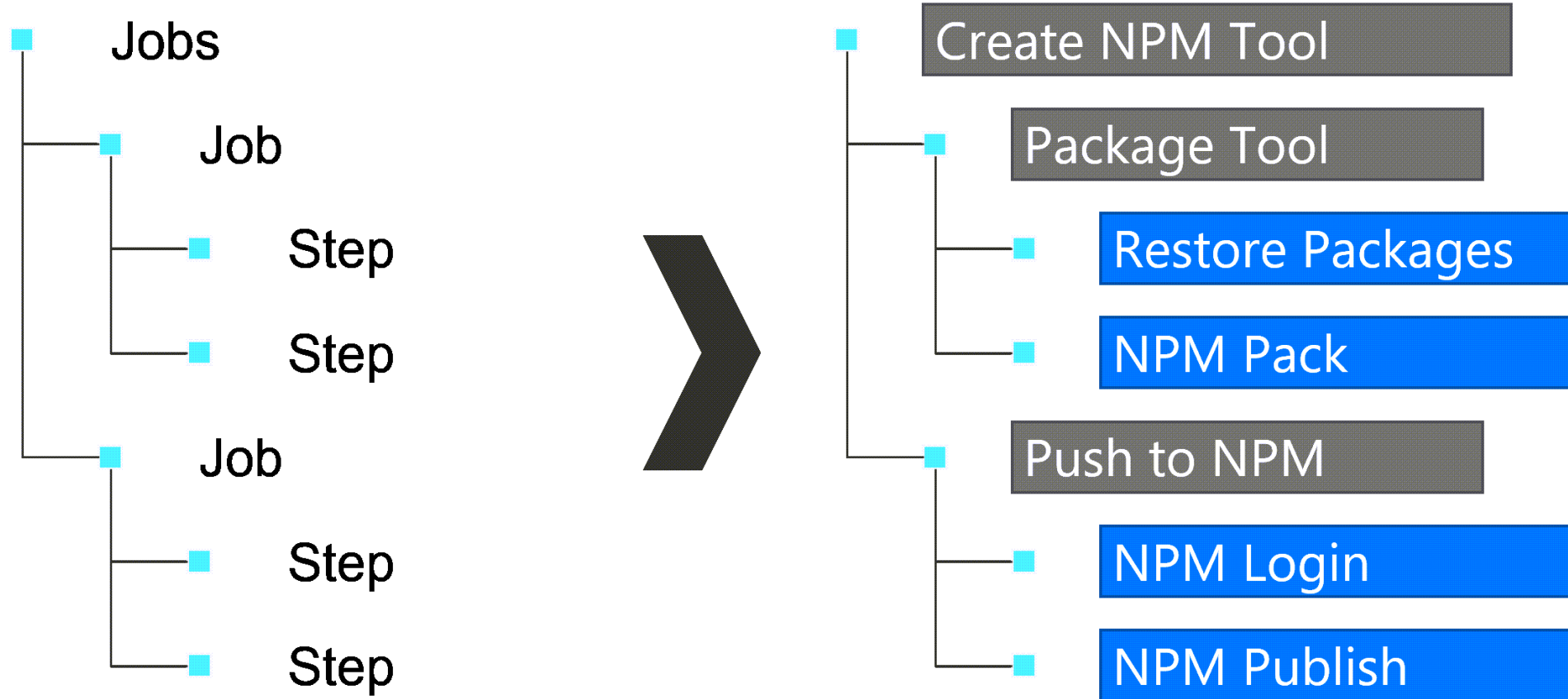
Deploy

Add Label

Anatomy of a workflow

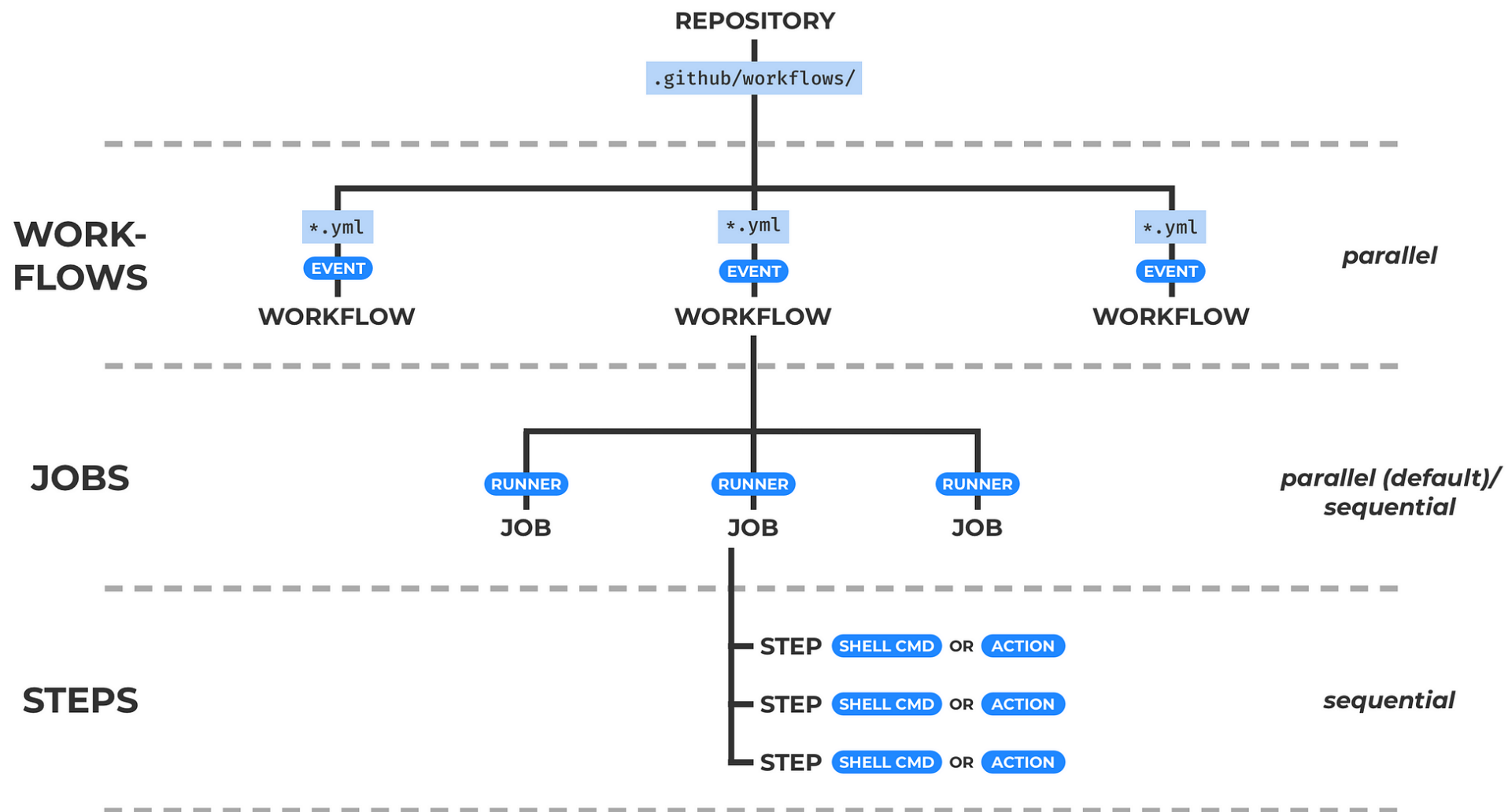


Example of a workflow





GitHub Actions



Workflows



- Types of Workflows
 - Automation
 - Pages
 - Continuous integration
 - Deployment
 - Security

Workflows - Automation



- Labeler
- Stale
- Greetings
- Manual workflow

Automation - Labeler



- Automatically label new pull requests based on the paths of files being changed or the branch name.

Automation - Stale



- The GitHub Stale Action is a community-maintained GitHub Action that helps automate the process of identifying and closing stale issues and pull requests in a GitHub repository.

Automation - Greetings



- The "greetings" GitHub Action automatically comments on pull requests from first-time contributors.

Automation - Manual workflow



- GitHub Actions has a feature called Manual Workflow Approval that pauses a workflow and requires manual approval before continuing.

Workflows – Pages



- You can use GitHub Pages to host a website about yourself, your organization, or your project directly from a repository on GitHub.com.

Triggers



- You can configure your workflows to run when specific activity on GitHub happens, at a scheduled time, or when an event outside of GitHub occurs.

Triggering workflows in multiple ways

There are many ways we can trigger GitHub workflows:

Repository event	push	issues	pull_request	pull_request_review	fork	...
	Triggered when someone pushes to the repo	Triggered by a variety of events related to issues	Triggered by a variety of events related to PRs	Triggered by a variety of events related to PR reviews (submitting, editing, deleting)	Triggered when your repository is forked	
	Triggered via the UI		Triggered via an API call	Triggered from another workflow		
	Triggered from the Actions tab in GitHub		Triggered via GitHub's REST API	Triggered from within another workflow		
Schedule	Runs as a cron job					

Runners



Virtual servers that execute jobs from workflows

GitHub-hosted (standard)

					
2 cores	7 GB	14 GB	3 cores	14 GB	14 GB
windows & ubuntu			mac		

- Managed service
- A VM is scoped to a job: steps share the VM, but jobs don't (by default, each job receives a clean VM instance)

Self-hosted

- Run workflows on (almost) any infrastructure of your choice
- Full control over the VM infrastructure
- It's not managed, meaning we need to take care of OS patching, software updates, among other ops tasks
- Can be added at the repository, organization, or enterprise level
- Jobs do not necessarily have to run on clean instances

Pro-tip

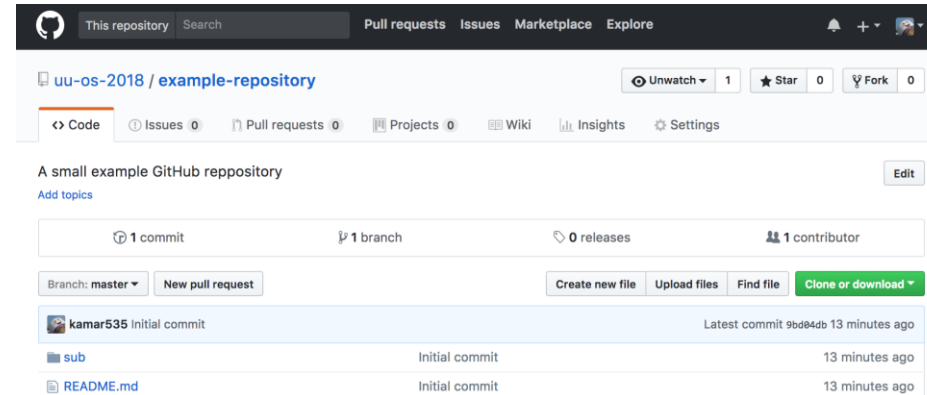
Keep the VM resources in mind, especially when running commands that rely on parallel execution (for example, running parallel jest tests).



Warning

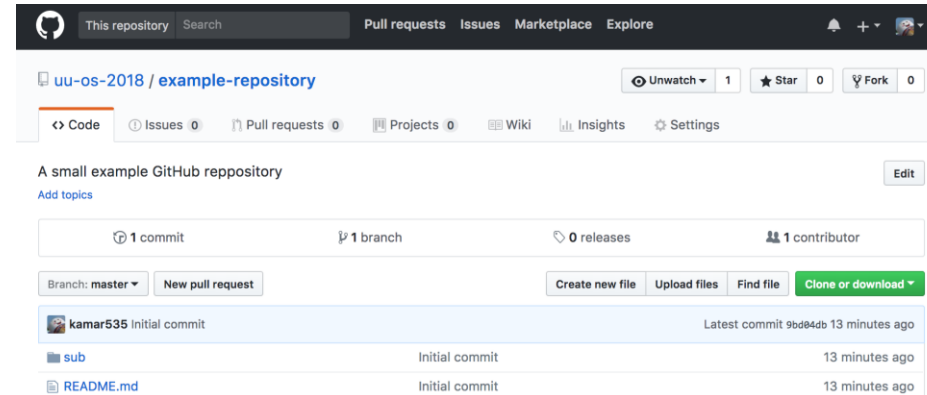
Do not use self-hosted runners in public repositories!

How GitHub Actions automate these workflows?



- when something happens IN or TO your repository
- automatic ACTIONS are executed in response

How GitHub Actions automate these workflows?



- when something happens IN or TO your repository

PR Created

Contr. joined

Other apps

Issue created

PR merged



- automatic ACTIONS are executed in response

How GitHub Actions automate these workflows?



1) LISTEN TO EVENT

PR Created

Contr. joined

GitHub **Events**

Other apps

Issue created

PR merged



2) TRIGGER WORKFLOW



GitHub **Events**

Sort

Action

Label

Action

Assign it

Action

Reproduce

Action

How GitHub Actions automate these workflows?



1) LISTEN TO EVENT

PR Created

Contr. joined

GitHub **Events**

Other apps

Issue created

PR merged



2) TRIGGER WORKFLOW



GitHub **Events**

Sort

Label

Assign it

Reproduce

Action

Action

Action

Action

How GitHub Actions automate these workflows?



1) LISTEN TO EVENT

PR Created

Contr. joined

GitHub **Events**

Other apps

Issue created

PR merged



Workflow

2) TRIGGER WORKFLOW



GitHub **Events**

Sort

Action

Label

Action

Assign it

Action

Reproduce

Action

GitHub Actions Marketplace



- The GitHub Actions Marketplace is a curated collection of pre-built actions that enable automation and streamline workflows within GitHub repositories, covering diverse tasks from testing and building to deploying and managing code.

- <https://github.com/marketplace?type=actions>

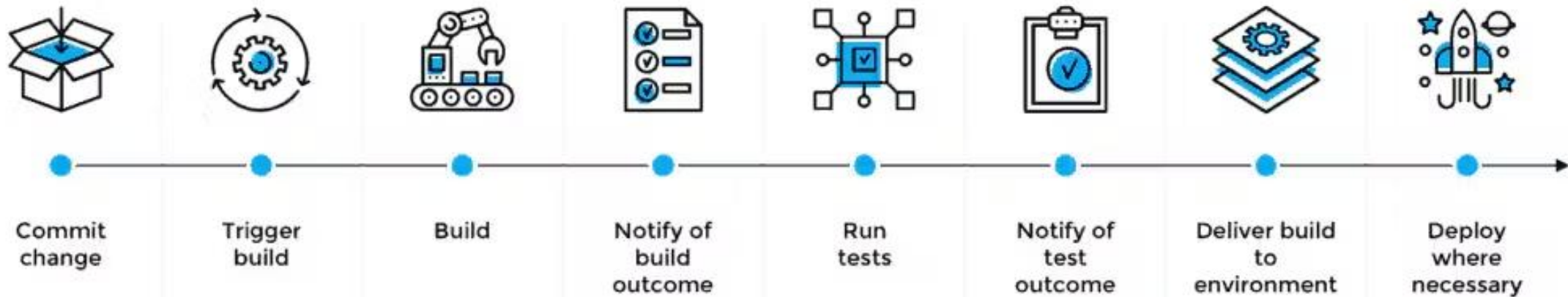
The screenshot displays the GitHub Actions Marketplace interface. On the left, a sidebar lists categories: Apps, Actions (selected), Categories, API management, Chat, Code quality, Code review, Continuous integration, Dependency management, Deployment, IDEs, Learning, and Localization. The main content area is titled 'Actions' and includes the subtitle 'An entirely new way to automate your development workflow.' It shows '300 results filtered by Actions'. A grid of action cards is displayed, each with an icon, name, description, and star count:

Action Name	Description	Stars
xo-action	JavaScript happiness style linter GitHub Action	20 stars
waka-box	Update a pinned gist to contain WakaTime stats	20 stars
validate-license-action	Action to validate that a repo contains a license of one of the allowed types	2 stars
vale-lint	Vale - linter for prose	3 stars
use-package-json-metadata-action	Use metadata defined in package.json as repository description, homepage and tags	-
update-git-branch-action	Update a git branch to the current commit	-
store-env	Stores environment variables in .profile	2 stars
shellcheck	Run shell check on ALL sh files in the repository	11 stars

GitHub Actions CI/CD pipeline



CI/CD Pipeline



Using GitHub Actions for CI/CD



Automates continuous integration (CI) processes like testing and code quality checks.



Facilitates continuous deployment (CD) by automating build, deploy, and release tasks.



Integrates seamlessly with GitHub repositories, simplifying workflow automation.

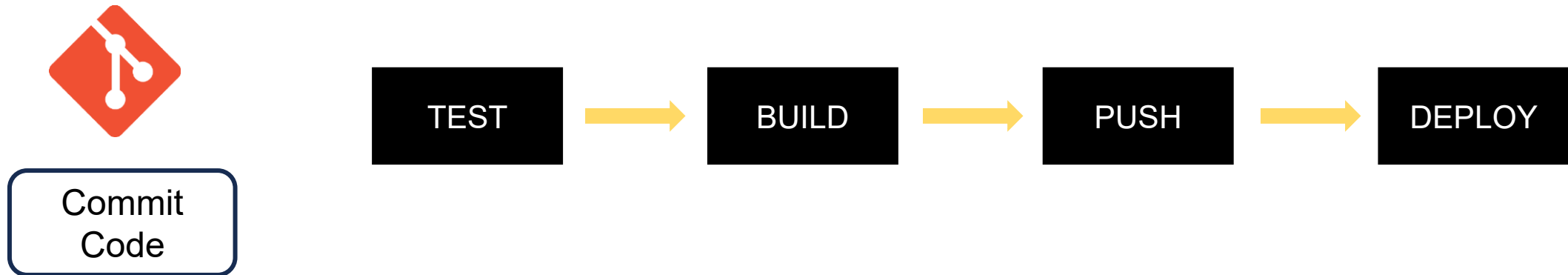


Enables version-controlled, scalable CI/CD pipelines directly within the GitHub ecosystem.

CI/CD with Github Actions



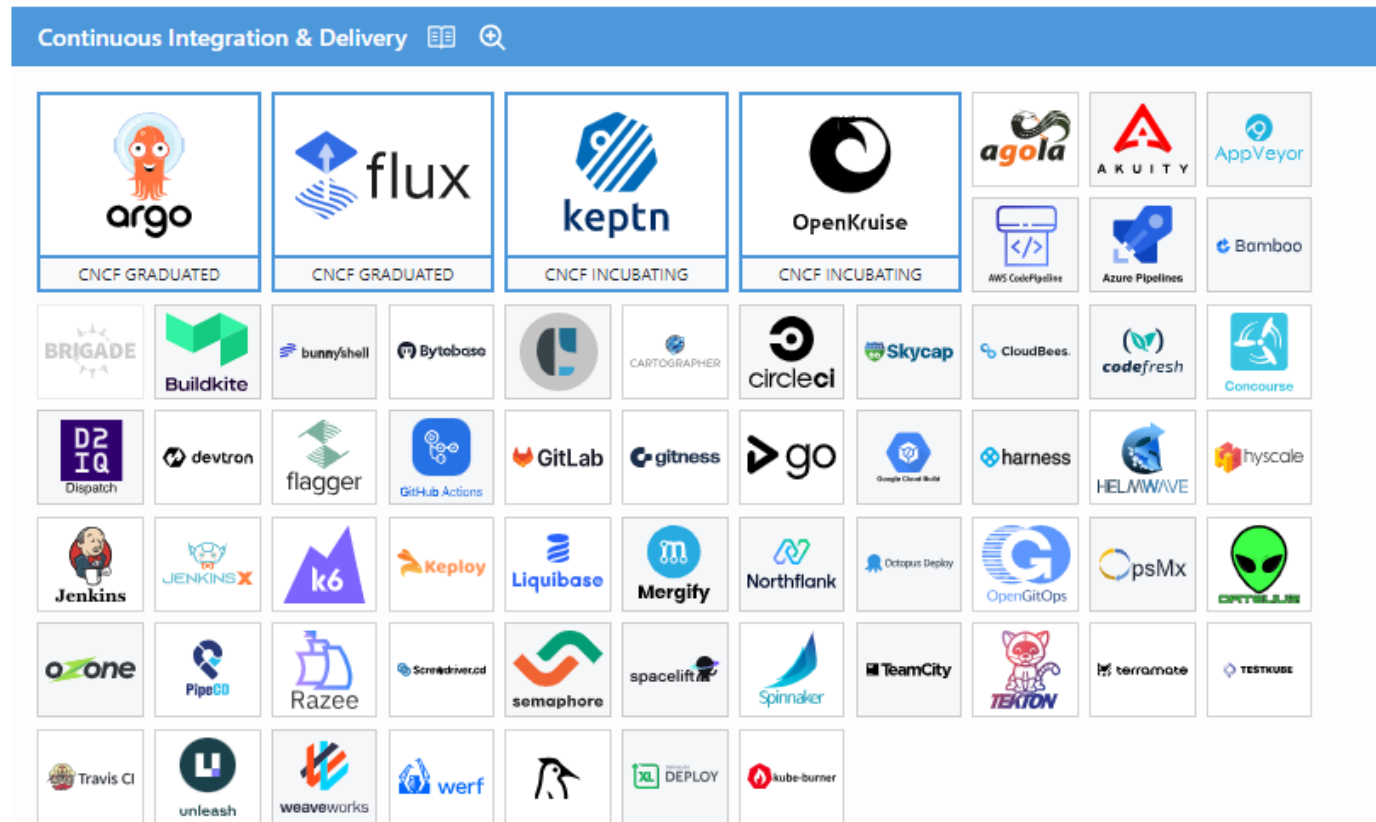
- most common workflow for your repository



CI/CD with Github Actions



Just another CI/CD tool?



CI/CD with Github Actions



Just another CI/CD tool?

- ✓ use same tool instead of third-party integration
- ✓ setup the pipeline is easy
- ✓ tool for developers

CI/CD with Github Actions



Why is the setup easier?

- integration with other technologies is important!

Pipeline



NodeJs App



Build Docker
image



Push to
Nexus Repo



Deploy to
DigitalOcean Server

CI/CD with Github Actions



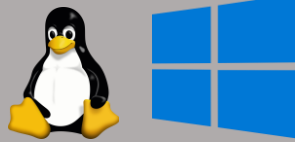
Why is the setup easier?

- integration with other technologies is important!

Pipeline



Java App
with Maven



Integration Tests
Linux & Windows



Build Docker
image



Push to
AWS Repo



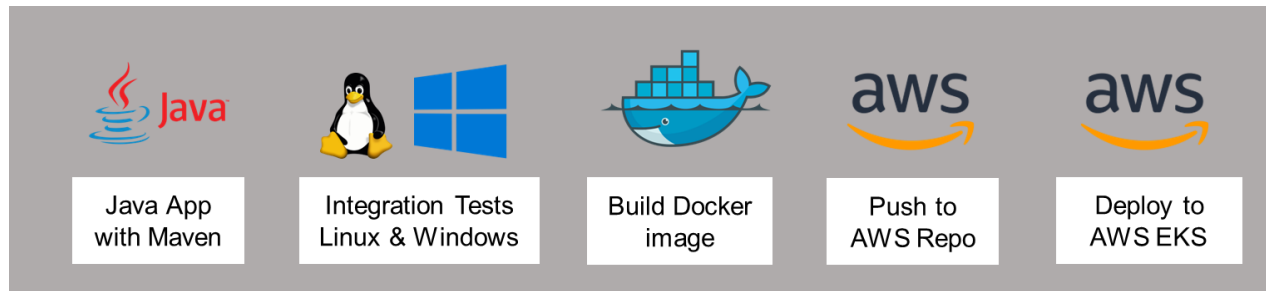
Deploy to
AWS EKS

CI/CD with Github Actions



Why is the setup easier?

- integration with other technologies is important!



Install Java

Install Docker

Install Maven

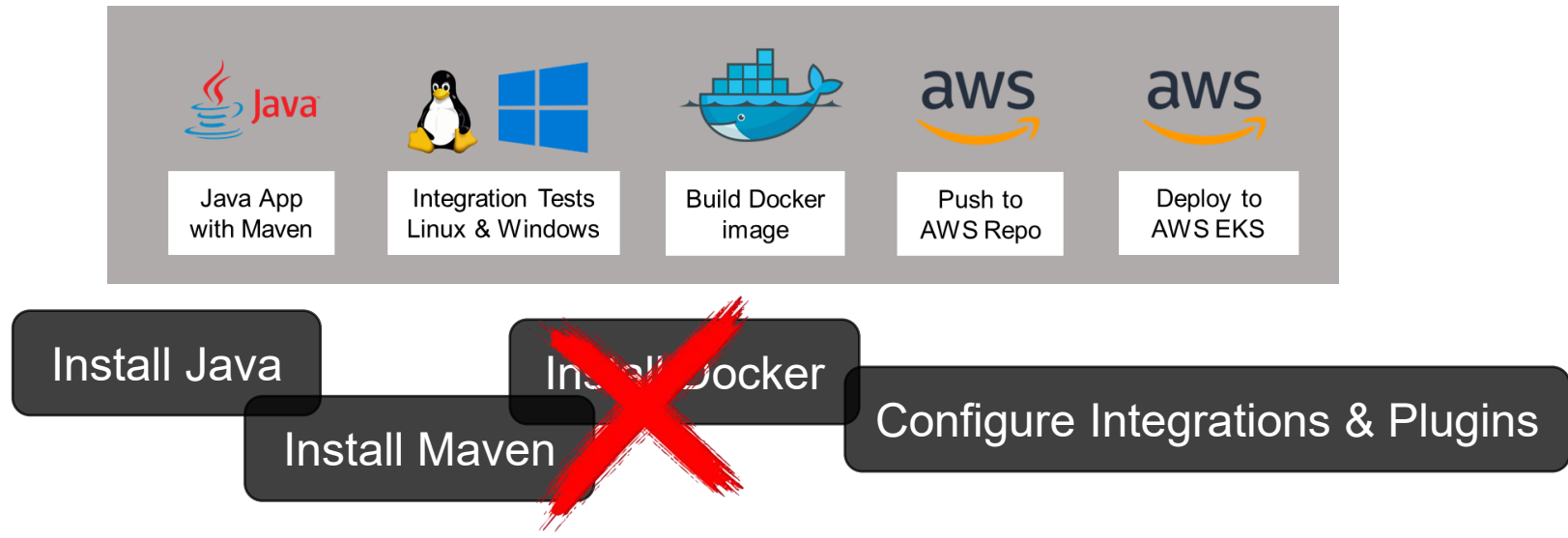
Configure Integrations & Plugins

CI/CD with Github Actions



Why is the setup easier?

- integration with other technologies is important!



CI/CD with Github Actions

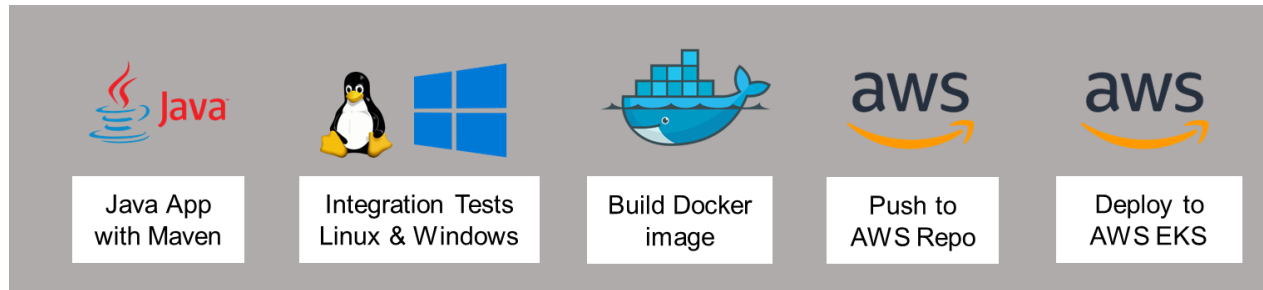


Why is the setup easier?

- integration with other technologies is important!



...give me an environment...

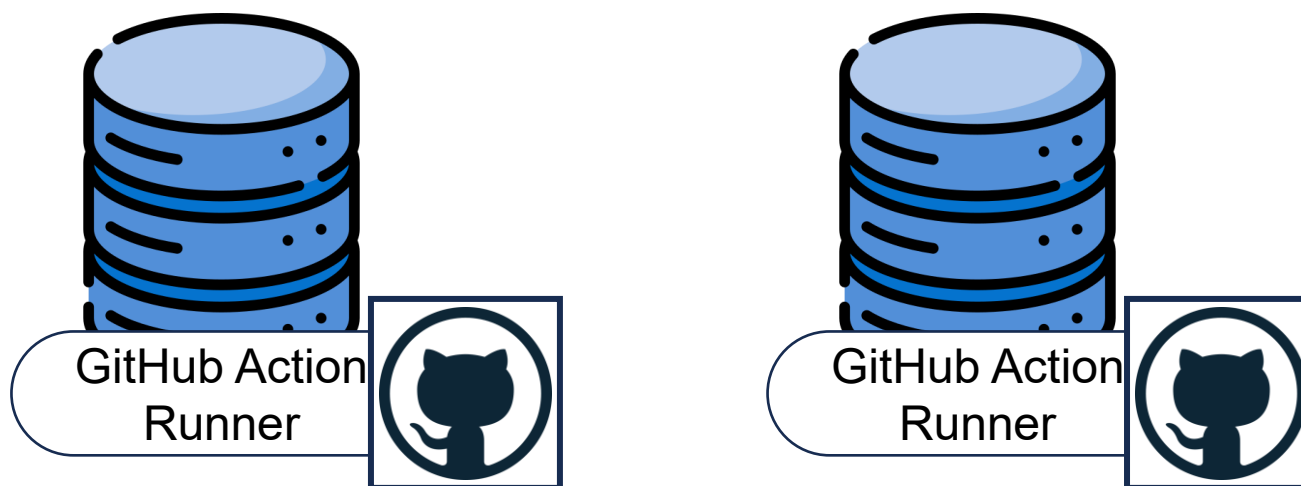


...with Node and Docker available
...with version specify
...simply connect to target and deploy



Where does this code run?

- managed by GitHub (BUT you can also host your own!)
- each job in a workflow runs in a fresh virtual environment



Actions



Custom applications to perform complex, frequently repeated tasks

```
name: My Java package
on: push

jobs:
  java-release:
    runs-on: ubuntu-latest
    steps:
      - name: Set up JDK
        uses: actions/setup-java@v2
        with:
          java-version: '11'
          distribution: 'adopt'
          cache: maven
      - name: Run Tests
        run: mvn test
```

Avoids repetitive and extensive commands

Prevents code duplication and reduces the chances of mistakes

Can be configured via the **with** key-value pair

Enables great flexibility and reusability

Can be combined with other steps

Can be used to encapsulate setup tasks before running other commands

We can create our own actions for public or private use

We are not restricted only to the actions available at the marketplace

Actions



- You can find actions in GitHub Marketplace

The screenshot shows the GitHub Marketplace interface. At the top, there's a search bar with the text 'Type / to search'. Below the search bar, the breadcrumb 'Marketplace / Search results' is visible. On the left sidebar, under 'Types', 'Actions' is selected. Under 'Categories', various options like 'API management', 'Chat', 'Code quality', etc., are listed. The main content area shows '22285 results for "sort:" filtered by Actions'. A list of actions is displayed, each with a play button icon, a title, a description, and star count. The actions listed are: 'Setup Java JDK' (1.4k stars), 'Setup Go environment' (1.3k stars), 'Setup .NET Core SDK' (896 stars), 'First interaction' (705 stars), 'Close Stale Issues', and 'Download a Build Artifact'.

Marketplace / Search results

Types

Apps

Actions

Categories

API management

Chat

Code quality

Code review

Continuous integration

Dependency management

Deployment

Deployment Protection Rules

IDEs

Learning

Localization

Mobile

Monitoring

Project management

Search: sort: Sort: Most installed/starred

Actions

An entirely new way to automate your development workflow.

22285 results for "sort:" filtered by Actions

Actions

Setup Java JDK

By actions

Creator verified by GitHub

Set up a specific version of the Java JDK and add the command-line tools to the PATH

1.4k stars

Setup Go environment

By actions

Creator verified by GitHub

Setup a Go environment and add it to the PATH

1.3k stars

Setup .NET Core SDK

By actions

Creator verified by GitHub

Used to build and publish .NET source. Set up a specific version of the .NET and authentication to private NuGet repository

896 stars

First interaction

By actions

Creator verified by GitHub

Greet new contributors when they create their first issue or open their first pull request

705 stars

Close Stale Issues

By actions

Creator verified by GitHub

Close issues and pull requests with no recent

Download a Build Artifact

By actions

Creator verified by GitHub

Download a build artifact that was previously

Event Filters



Specify under which conditions a specific event triggers our workflow

Most triggers can be configured to specify when they should run.

push event

branches

Specifies which branches must match in order for the workflow to execute

branches_ignore

Specifies which branches must not match in order for the workflow to execute

tags

tags_ignore

paths

paths_ignore

.

```
name: My package workflow
on:
  push:
    branches:
      - main
      - 'releases/**'
    paths-ignore:
      - 'docs/**'
```

If multiple filters are specified, all of them must be satisfied for the workflow to run!

Activity Types



Specify which types of certain triggers execute our workflow

Many triggers have multiple activity types we can leverage.

pull_request event	
opened	synchronize
Runs the workflow whenever a PR is opened.	Runs the workflow whenever a new commit is pushed to the HEAD ref of the PR.
closed	assigned
labeled	edited
.	

```
name: My package workflow on:
  pull_request:
    types: [opened, synchronize]
    branches:
      - main
      - 'releases/**'
```

We can specify one or more activity types

Activity types and event filters can be combined when needed

Workflow Contexts



Access information about runs, variables, jobs, and much more

GitHub provides multiple sources of data in different contexts so that we can easily provide all the necessary information to our workloads

<code>github</code> ■ Commit SHA ■ Eventname ■ Ref of branch or tag triggering the workflow	<code>env</code> Contains variables that have been defined in a workflow, job, or step. Changes based on which part of the workflow is executing.	<code>inputs</code> Contains input properties passed via the keyword <code>with</code> to an action, to a reusable workflow, or to a manually triggered workflow.	<code>vars</code> Contains custom configuration variables set at the organization, repository, and environment levels.
<code>secrets</code>	<code>matrix</code>	<code>needs</code>	<code>.</code>

Expressions



Use dynamic values and expressions in your workflows

- Can be used to reference information from multiple sources within the workflow
- Must use the `${{ <expression> }}` syntax
- Can be any combination of:

Literal values	Context values	Functions
Strings, numbers, booleans, null.	Values passed via the many workflow contexts.	Built-in functions provided by GitHub Actions.

- Support the use of functions and operators such as `!`, `<`, `>`, `!=`, `&`, `|`, and many others.

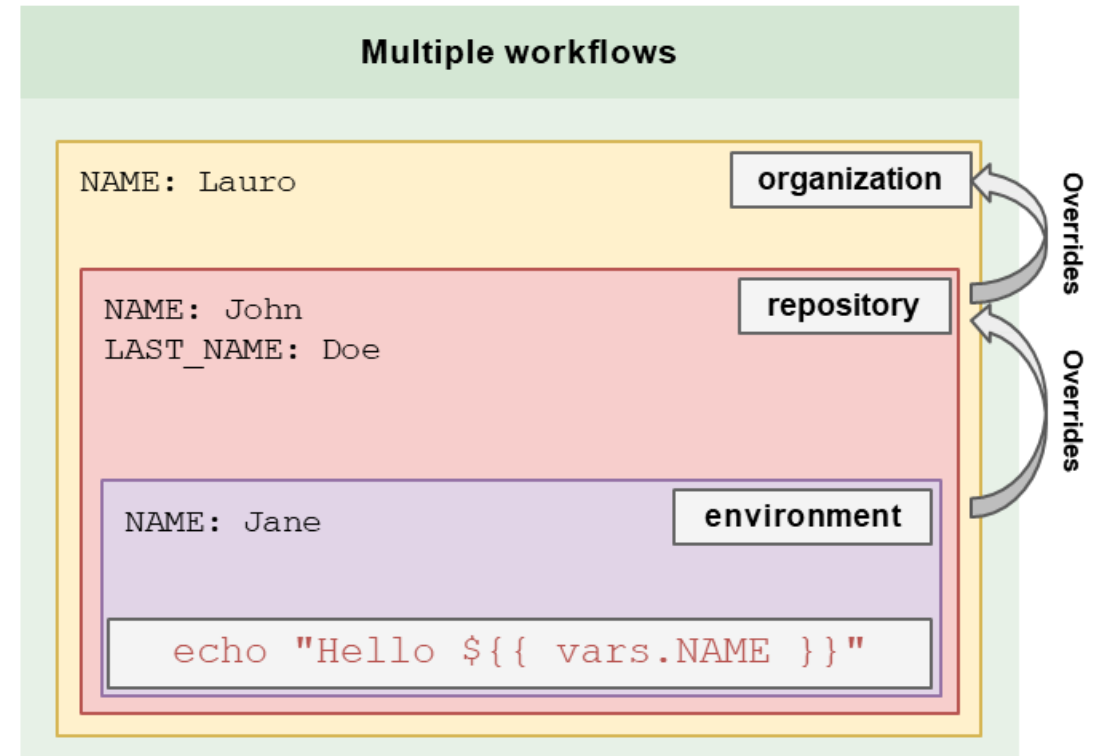
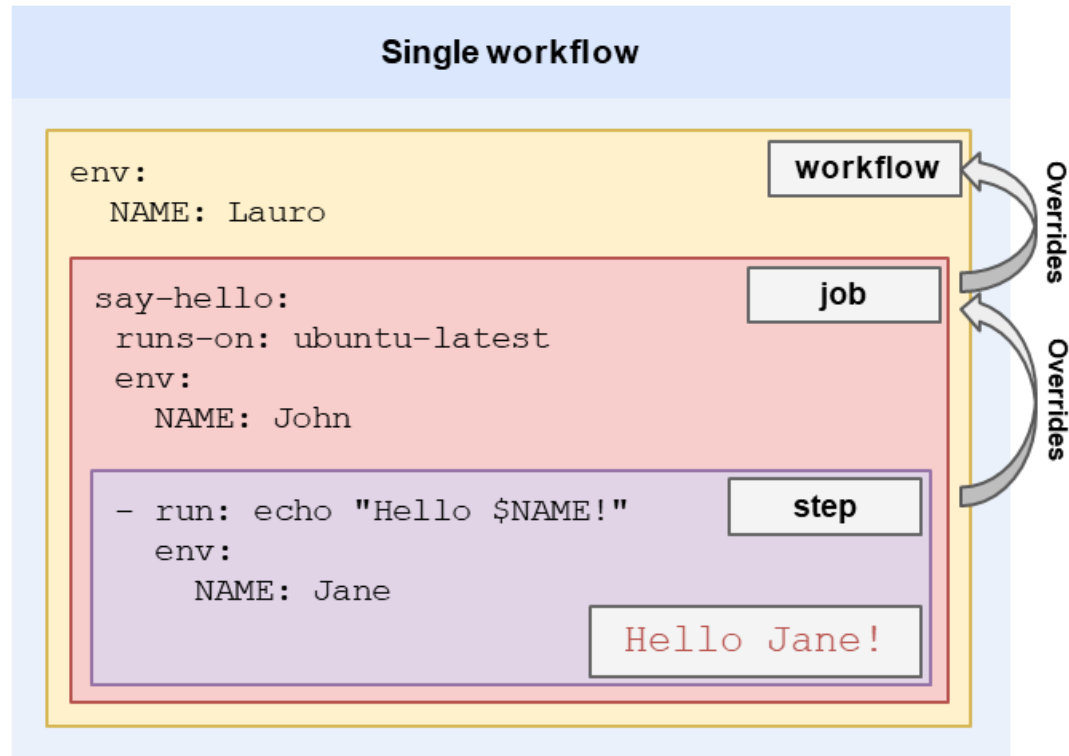
```
name: My workflow
on: [push, pull_request]

jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
      - if: ${ github.event_name } = "push"
        run: |
          echo "Triggered by a push event"
          echo "Running on ${ github.ref_name }"
```

Variables



Set and reuse non-sensitive configuration information



Functions



Out-of-the-box functions to model complex behavior

General purpose functions

Provides a set of utility functions to interact with data from multiple contexts and model more complex behavior, such as more advanced control of the workflow and job execution.

`contains()`

`startsWith()`

`endsWith()`

`fromJSON()`

`toJSON()`

`.`

Status check functions

Provides a set of functions that allow using the status of the workflow, previous jobs or steps to define whether a certain job or step should be executed.

`success()`

`failure()`

`always()`

`cancelled()`

Use-case

`fromJSON()` can be used to convert variables from `string` to different data types.



Use-case

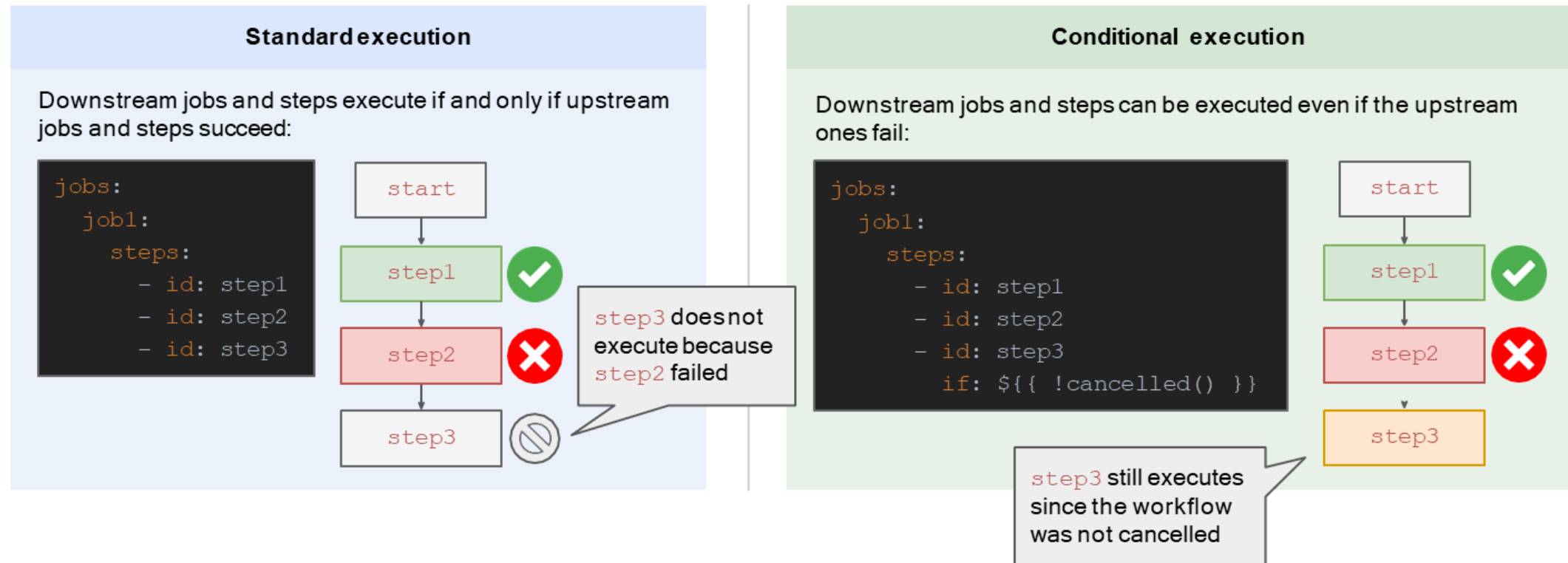
`!cancelled()` can be used to execute jobs or steps even if previous jobs or steps failed, but to prevent execution in case the workflow is cancelled.

Controlling the Execution Flow



Execute jobs and steps conditionally, and set dependencies between jobs

Conditional job execution via the `if` key:

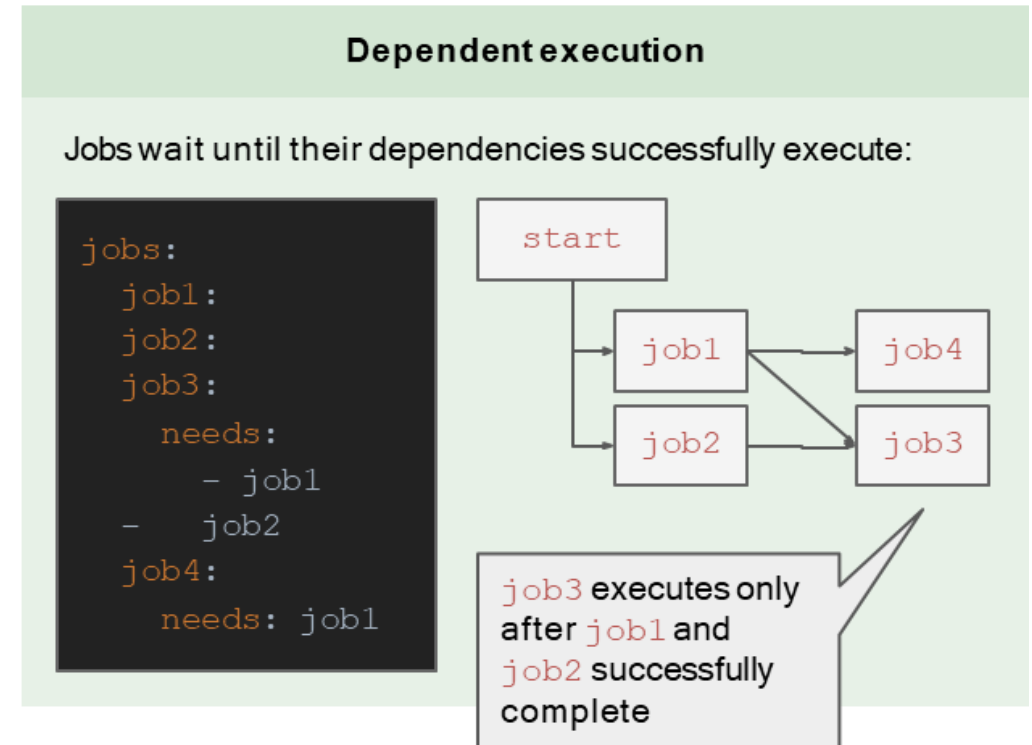
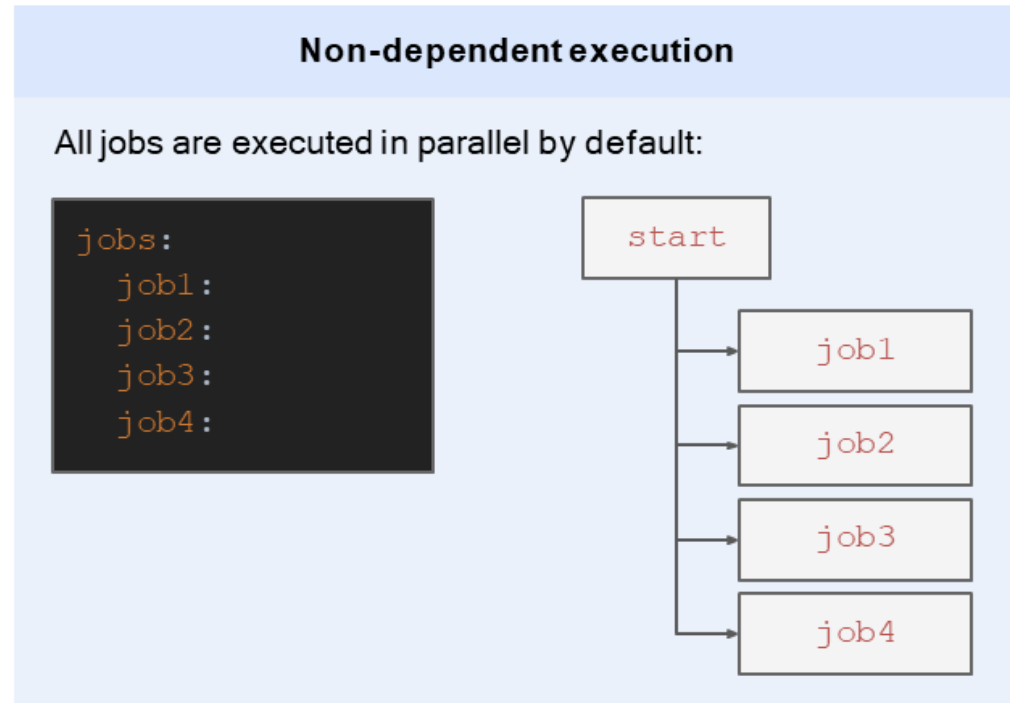


Controlling the Execution Flow



Execute jobs and steps conditionally, and set dependencies between jobs

Sequential job execution via the `needs` key:



Inputs



Inputs enable us to request specific information from the workflow or action caller and use this information at runtime.

GitHub Action

```
inputs:
  url:
    description: '.'
    required: true
  max-trials:
    description: '.'
    required: false
    default: '60'
```

Inputs must be defined in the definition of reusable workflows or custom actions so that they can be correctly used.

Caller

```
jobs:
  job1:
    steps:
      - name: Ping URL
        uses: ping-url-example@v1
        with:
          url: https: /www.google.com
          max-trials: 10
```

Use-case

Provide information such as target runtime version (when setting up node, for example), caching keys, among others.

Outputs



Output data from jobs for later usage

```
jobs:
  welcome:
    runs-on: ubuntu-latest
    outputs:
      name: ${ steps.step1.outputs.NAME }
    steps:
      - id: step1
        run: echo "NAME=Lauro" > "$GITHUB_OUTPUT"
  goodbye:
    runs-on: ubuntu-latest
    needs: welcome
    steps:
      - run: echo "Bye, ${ needs.welcome.outputs.name }"
```

Outputs



Output data from jobs for later usage

- 1 Define a step ID for the steps that produce outputs

```
- id: step1
```

Outputs



Output data from jobs for later usage

```
- id: step1  
  run: echo "NAME=Lauro" > "$GITHUB_OUTPUT"
```

- 1 Define a step ID for the steps that produce outputs
- 2 Echo key-value pairs to the `$GITHUB_OUTPUT` variable

Outputs



Output data from jobs for later usage

```
outputs:
  name: ${ steps.step1.outputs.NAME }
steps:
  - id: step1
    run: echo "NAME=Lauro" > "$GITHUB_OUTPUT"
```

- 1 Define a step ID for the steps that produce outputs
- 2 Echo key-value pairs to the `$GITHUB_OUTPUT` variable
- 3 Mention the outputs in the `outputs` section of the job

Outputs



Output data from jobs for later usage

```
jobs:
  welcome:
    runs-on: ubuntu-latest
    outputs:
      name: ${ steps.step1.outputs.NAME }
    steps:
      - id: step1
        run: echo "NAME=Lauro" > "$GITHUB_OUTPUT"
  goodbye:
    runs-on: ubuntu-latest
    needs: welcome
```

- 1 Define a step ID for the steps that produce outputs
- 2 Echo key-value pairs to the `$GITHUB_OUTPUT` variable
- 3 Mention the outputs in the `outputs` section of the job
- 4 List the job as a dependency of jobs that need the output

Outputs



Output data from jobs for later usage

```
jobs:
  welcome:
    runs-on: ubuntu-latest
    outputs:
      name: ${ steps.step1.outputs.NAME }
    steps:
      - id: step1
        run: echo "NAME=Lauro" > "$GITHUB_OUTPUT"
  goodbye:
    runs-on: ubuntu-latest
    needs: welcome
    steps:
      - run: echo "Bye, ${ needs.welcome.outputs.name }"
```

- 1 Define a step ID for the steps that produce outputs
- 2 Echo key-value pairs to the `$GITHUB_OUTPUT` variable
- 3 Mention the outputs in the `outputs` section of the job
- 4 List the job as a dependency of jobs that need the output
- 5 Access the output via the `needs` context

Managing Workflows



- Workflows are a configurable automated process that will run one or more jobs.
- Workflows are defined by a YAML file checked in to your repository and will run when triggered by an event in your repository.
- A workflow must contain one or more events that will trigger the workflow, and one or more jobs, each of which will execute on a runner machine and run a series of one or more steps.
- Each step can either run a script that you define or run an action, which is a reusable extension that can simplify your workflow.

Managing Dependencies



- To cache dependencies for a job, you can use GitHub's cache action.
- The action creates and restores a cache identified by a unique key.
- Caching allows your workflow to store data for future uses, limiting the time spent downloading and making requests.
- If you use a package manager like npm and on every run, you need to get the same dependency, you should cache it.

Caching



Speed up workflow runs by caching stable files

Caching allows us to store files and later retrieve them based on a key. Workflows can access the cache from their branch or from the default branch.



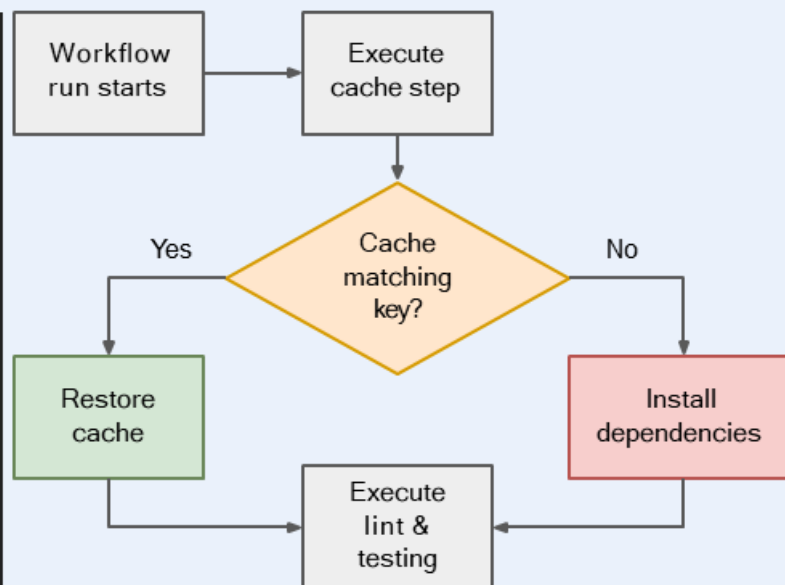
Use-case

Cache dependencies to speed up execution by avoiding always downloading and installing them.

Caching process

```
steps:
- name: Setup Java

uses:actions/setupjava@v4
with:
  distribution: 'adopt'
  java-version: '11'
  cache: 'maven'
- name: Setup Java
  uses:mvn dependency
```



Artifacts



Share data between jobs and store data after workflows have completed

Artifacts

- Stored for up to 90 days
- Managed via two actions
 - upload-artifact
 - download-artifact
- Recommended when the stored files are likely to be accessed outside the workflow, for example:
 - Build outputs
 - Test results
 - Logs



Caching

- Stored for up to 7 days
- Managed via a single action
 - cache
- Recommended when the stored files are likely to be accessed only within the workflow, for example:
 - Build dependencies



Use-case

Uploading and downloading build outputs for deployments

Optimizing Workflows



- You can manage workflow concurrency, implement conditional statements, and augment workflows with in-house scripts to streamline and accelerate your development pipeline.
- You can analyze your current workflow to see if you can divide it into different jobs.
- Jobs as opposed to steps can be run in parallel by different machines (runners).
- You can select a faster runner.
- The strategy matrix is a very interesting, yet powerful syntax in GitHub action. The matrix allows you to test many (up to 256) version configurations for the same job.